

# Engineering-First Software Architecture

## Non-Technical Briefing Document

---

### What This Document Covers

This brief explains a proven approach to building engineering calculation software that remains **clear, reliable, and maintainable** for years. It's written for engineers, project managers, and technical leads who need to understand the philosophy without diving into code.

---

### The Core Problem

Engineering software often becomes difficult to maintain because:

1. **Software trends change faster than engineering needs**
2. **Complex architectural patterns obscure simple calculations**
3. **UI frameworks get mixed with mathematical logic**
4. **"Enterprise" patterns add unnecessary complexity to scientific tools**

This creates a mismatch: engineering calculations need to be **traceable and stable**, but software architecture often prioritizes **flexibility and extensibility**.

---

### The Solution: Engineering-First Architecture

Think of this approach as designing software the way you'd design a calculation workflow:

#### Traditional Engineering Workflow

Physical Model → Equations → Hand Calculations → Results → Verification

#### Our Software Workflow

Physical Model → Equations → Code → Results → Verification

**Key principle:** Keep the distance between physics and code as short as possible.

---

### Core Philosophy

#### 1. Treat Software Like a Reference Manual

Your calculation code should read like a textbook:

- Clear variable names
- Visible formulas
- Explicit steps
- No hidden magic

**Analogy:** If you open a mechanics textbook, you immediately see the equations and understand what they do. Your code should work the same way.

## 2. Separate What Changes from What Doesn't

**Rarely changes:**

- Physics equations
- Mathematical models
- Core calculations

**Changes frequently:**

- User interfaces (desktop vs. web vs. mobile)
- Input/output formats (CSV, Excel, databases)
- Display preferences

**Solution:** Keep calculations completely separate from UI and data storage.

## 3. Explicitness Over Cleverness

**Bad example:**

"This uses a factory pattern with dependency injection  
to dynamically resolve calculation strategies..."

(Translation: The math is hidden behind multiple layers.)

**Good example:**

"This calculates bending stress as moment divided by inertia."

(Translation: The math is visible.)

---

## What This Architecture Looks Like

Think of it as three distinct layers:

### **Data Layer (What you're working with)**

- Body properties
- Material characteristics
- Test measurements
- Configuration parameters

**Characteristics:**

- Pure data, no behavior
- Strongly typed (compiler catches errors)
- Immutable (values don't change accidentally)



## Calculation Layer (What you're computing)

- Equations
- Calculations
- Analyses
- Numerical integration

### Characteristics:

- Pure mathematics
- No UI references
- No database connections
- Completely testable



## Interface Layer (How you interact)

- Console applications
- Desktop windows
- Web interfaces
- File import/export

### Characteristics:

- Thin shells around calculations
- Easily replaceable
- Don't affect core math

---

## Practical Benefits

### For Engineers:



#### Can verify calculations by reading code

- No hidden layers between you and the math
- Formulas are explicit and traceable
- Easy to compare with reference calculations



#### Can modify calculations confidently

- Changes don't ripple through unrelated code
- Testing is straightforward
- Impact is contained and predictable

### For Project Managers:



#### Reduced long-term maintenance costs

- Code doesn't become "legacy" quickly
- New engineers can understand it rapidly
- Less dependency on specific frameworks

#### ✓ Flexibility in delivery

- Same calculation engine works for desktop, web, or console
- Can add new interfaces without rewriting math
- Easy to create different versions for different users

### For Organizations:

#### ✓ Knowledge retention

- Code documents itself
- Less reliance on specific developers
- Easier onboarding for new team members

#### ✓ Regulatory compliance

- Calculations are traceable
- Verification is straightforward
- Audit trail is clear

---

## What This Is NOT

### ✗ NOT Enterprise Architecture

This isn't Microsoft's or Oracle's approach to building business systems with thousands of users and complex workflows.

### ✗ NOT a Framework

You're not locked into a vendor's way of doing things. This is a pattern you control.

### ✗ NOT Oversimplified

This approach handles complex calculations just fine. What it avoids is *unnecessary* complexity in the architecture.

### ✗ NOT Anti-Modern

This uses modern C# features (strong typing, pattern matching, modern syntax). It's just selective about which features add value.

---

## Comparison with Other Approaches

| Aspect   | This Architecture        | MVC/MVVM              | Clean Architecture       |
|----------|--------------------------|-----------------------|--------------------------|
| Best for | Engineering calculations | Web apps, interactive | Large enterprise systems |

| Aspect            | This Architecture | MVC/MVVM | Clean Architecture |
|-------------------|-------------------|----------|--------------------|
| UIs               |                   |          |                    |
| Complexity        | Low               | Medium   | High               |
| Learning curve    | Gentle            | Moderate | Steep              |
| Code visibility   | High              | Medium   | Low                |
| Team size         | 1-10              | 5-50     | 20+                |
| Verification ease | Easy              | Moderate | Difficult          |

## When to Use This Architecture

### Excellent fit:

- Structural analysis tools
- Hydrodynamic calculations
- Thermal modeling
- Simulation engines
- Scientific computing
- Batch processing systems

### Possible fit (with modifications):

- Engineering tools with moderate UI complexity
- Analysis dashboards
- Calculation servers

### Poor fit:

- Social media platforms
- E-commerce systems
- Real-time collaborative tools
- Systems where UI is more complex than calculations

## Growth Path

### Phase 1: Proof of Concept

- Single executable
- Console interface
- Core calculations established

**Timeline:** Weeks

### Phase 2: Production Tool

- Separate calculation library
- Professional desktop UI

- File import/export
- Basic testing

**Timeline:** Months

### **Phase 3: Multiple Interfaces**

- Desktop application
- Web interface
- API for automation
- Comprehensive testing

**Timeline:** Quarters

**Key advantage:** Core calculations remain unchanged through all phases.

---

## **Risk Management**

### **What We Avoid:**

- ✗ **Over-engineering** - No abstract factories for single implementations
- ✗ **Premature optimization** - Add complexity only when needed
- ✗ **Framework lock-in** - Core calculations are framework-independent
- ✗ **Hidden behavior** - Everything is explicit and traceable

### **What We Embrace:**

- ✓ **Simplicity** - Code should be boring and obvious
  - ✓ **Explicitness** - Clear is better than clever
  - ✓ **Stability** - Calculations don't change with UI trends
  - ✓ **Verifiability** - Anyone can audit the math
- 

## **Cost Considerations**

### **Initial Development:**

- **Similar to traditional approaches**
- Possibly faster for calculation-heavy tools
- Less time spent on architectural decisions

## Maintenance (Where Savings Occur):

- **Significantly lower** than framework-heavy approaches
- Easier to modify calculations
- Faster onboarding of new engineers
- Less vulnerability to framework changes

## Long-Term (5-10 Years):

- **Much more stable** than alternatives
  - No major architectural rewrites needed
  - UI can be modernized without touching calculations
  - Knowledge transfer is straightforward
- 

## Decision Framework

Ask these questions about any proposed architectural addition:

1. **Does this make the calculation clearer?**
    - If no → Don't add it
  2. **Will I understand this in 6 months without documentation?**
    - If no → Simplify it
  3. **Is this solving a problem we have today?**
    - If no → Wait until we actually need it
  4. **Does this hide the math or expose it?**
    - If it hides → Reconsider the approach
- 

## Success Metrics

How to measure if this architecture is working:

### Technical Metrics:

-  New engineers can read calculations within days, not weeks
-  Calculation modifications don't break UI
-  UI changes don't require calculation retesting
-  Code reviews focus on math correctness, not architecture

### Business Metrics:

-  Lower maintenance costs over time
-  Faster feature delivery
-  Easier regulatory compliance

-  Better knowledge retention during turnover
- 

## Common Questions

### "Isn't this just old-fashioned procedural programming?"

**Short answer:** No. It's modern C# with strong typing, pattern matching, and excellent tooling—just without unnecessary architectural ceremony.

**Analogy:** This is like using a modern calculator instead of an abacus, but not insisting on a supercomputer for basic arithmetic.

### "Won't we need to refactor this later for enterprise use?"

**Short answer:** Only the UI and infrastructure layers. Core calculations remain stable.

**Analogy:** You might renovate the lobby and offices of a building, but the structural engineering doesn't change.

### "What if requirements change?"

**Short answer:** This architecture handles change well because concerns are separated clearly.

#### What changes easily:

- UI appearance and workflow
- Data storage format
- Input/output methods

#### What stays stable:

- Physics equations
- Mathematical models
- Core algorithms

### "How does this compare to what others are doing?"

**Reality check:** Many long-lived engineering tools (ANSYS, MATLAB core, SolidWorks calculations) follow similar principles internally, even if they use modern UI frameworks externally.

**This approach has decades of proven success in scientific computing.**

---

## Summary

This architecture is designed for **engineering realities**:

-  Calculations must be **verifiable**
-  Code must be **readable** by engineers, not just programmers
-  Tools must **survive** UI trends and framework changes
-  Maintenance costs must be **predictable** and low
-  Knowledge must be **transferable** between team members

**One-sentence summary:**

A proven approach to building engineering software that keeps calculations clear, separates concerns naturally, and remains maintainable for years without architectural complexity.

---

## References and Further Reading

- **Functional Core, Imperative Shell** - Gary Bernhardt
  - **Data-Oriented Design** - Scientific computing literature
  - **Engineering Software as a Service** - Best practices from scientific computing
  - C# language documentation - Modern features for engineering
- 

*This architecture prioritizes engineering needs over software trends—exactly what calculation tools require for long-term success.*